

Model performance evaluation

Training / test - evaluation

80 - 20

70 - 30

Validation = Dev

Development

Cross validation
k fold

} divide the data in k (amount of times) folds

train in $k-1$ of them

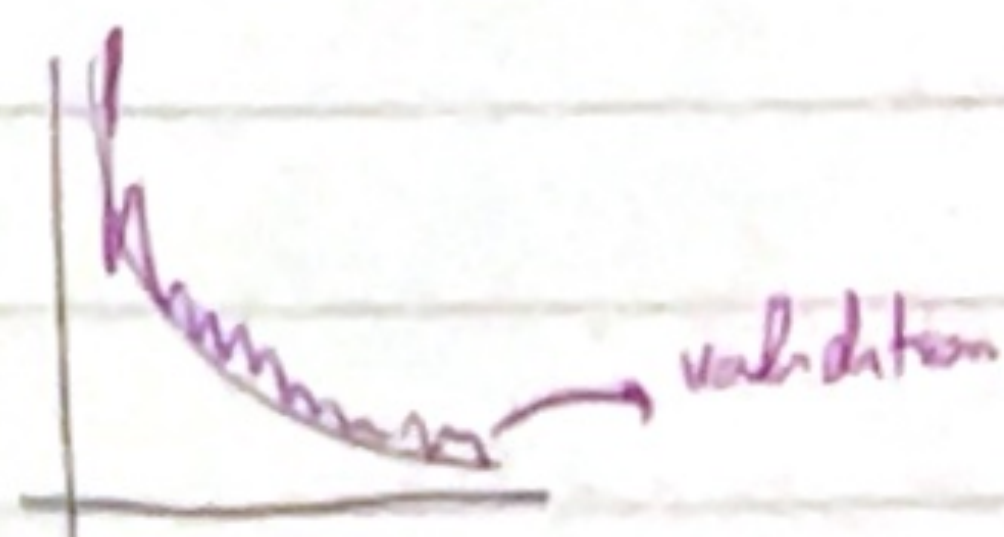
→ Stop before overfitting

when model fits to more than just the signal that is useful about the features in a dataset and also begins to memorize other stuff

Underfitting

can't capture patterns in data
model / algorithm are too simple for complex data

learning curves → plots of model performance over time



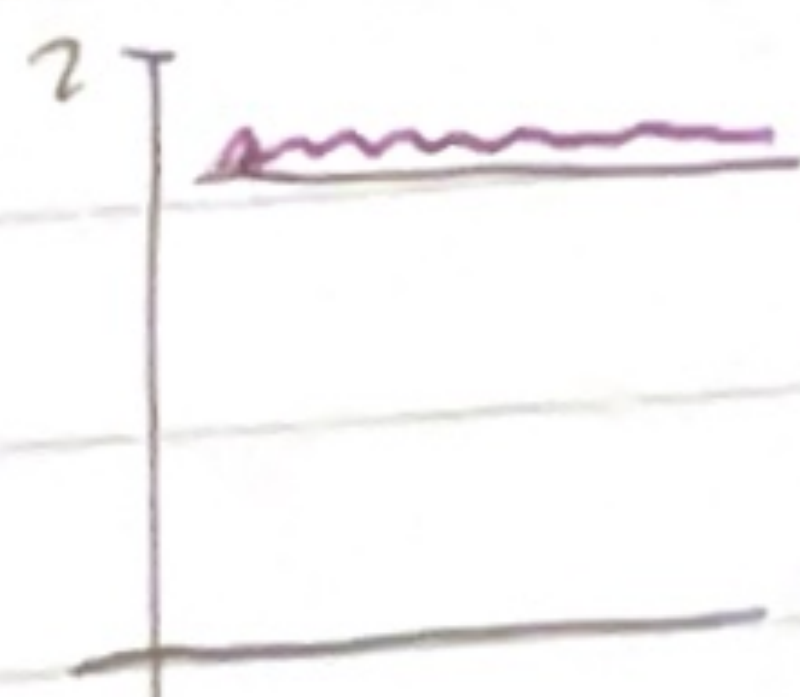
plot both curves → useful for debugging

→ Computing loss

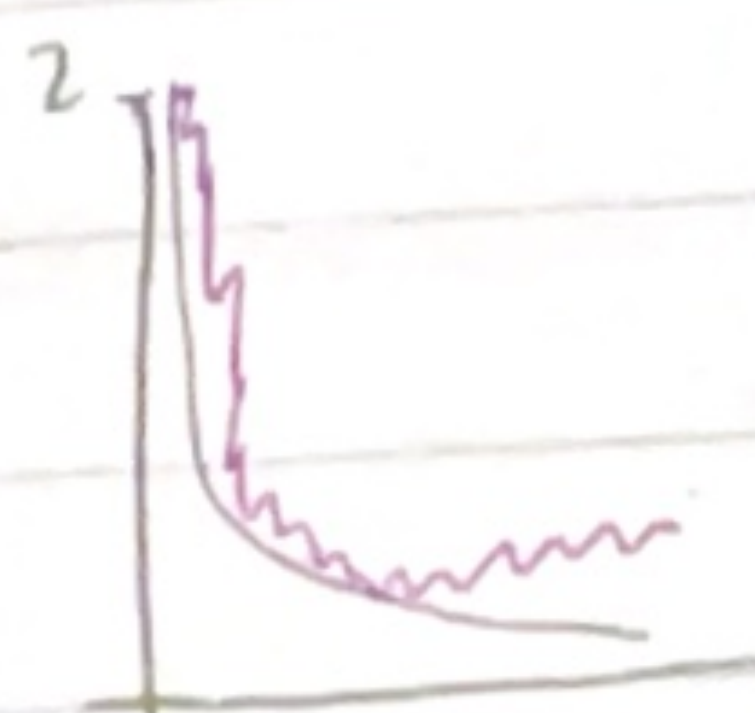
- training set → free / doesn't take time → done frequently
- validation → costly / takes time → gradually

epoch ← After every pass through training data

Under fitting



over fitting

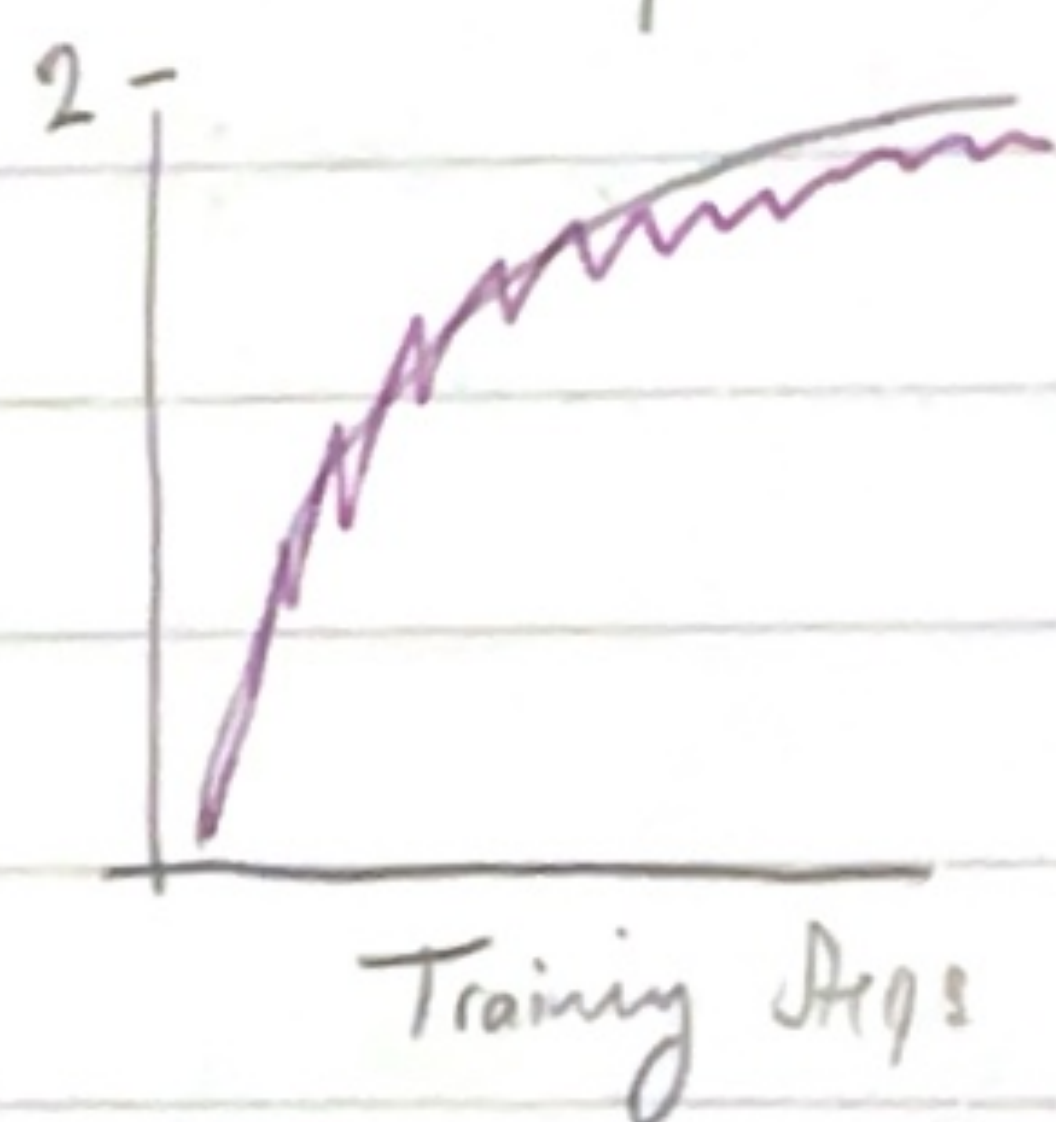


⇒ Not learning from model → performs poorly $\begin{matrix} \nwarrow T \\ \searrow V \end{matrix}$ sets

→ Good fit → Reaching a plateau

→ Generalization gap → Training loss - Validation loss

Model Accuracy Curve



STRATEGIES

DNN Models $\rightarrow$ universal ^{function} approximators

⚠ if model doesn't have enough info $\rightarrow$ E.g. medical imager } reduce pixels
|
miss info

Strategies for Underfitting

$\rightarrow$ $\uparrow$ time

$\rightarrow$ $\uparrow$ parameters $\rightarrow$ $\uparrow$ complexity e.g. deep neural network
 $\uparrow$ layers or $\uparrow$ neurons / layers

$\downarrow$ Overfitting $\rightarrow$ Regularization

$\rightarrow$ Weight decay L1 / L2 $\rightarrow$ penalise model for using
 $\uparrow$ parameters
 $\downarrow$ "

$\rightarrow$ Dropout $\rightarrow$ drops out values randomly

$\rightarrow$ model builds redundancy $\rightarrow$ $\downarrow$ complexity

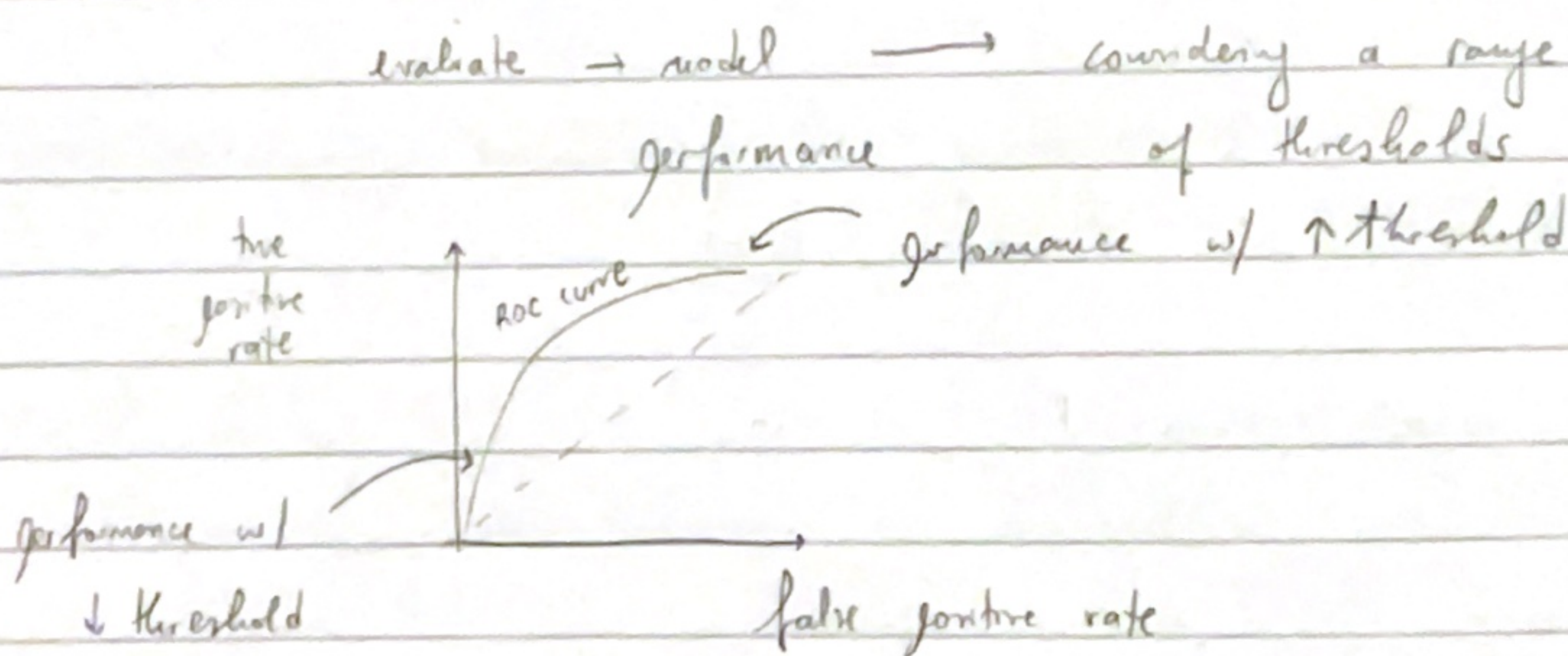
$\rightarrow$ Data augmentation - transforms the data $\rightarrow$ so the model learns

$\uparrow$ generalisable features
rather than
memorising

Statistical Approaches to Model Evaluation

threshold to convert probability to $\begin{matrix} \nearrow 1 \\ \searrow 0 \end{matrix}$
 $\downarrow$
 typically 0.5

Receiving operating Characteristic (ROC) Curves



ROC curves $\rightarrow$ each category vs. another

CONFUSION MATRIX

		predicted values	
		positive	negative
Actual values	positive		
	negative		

eg

actual		predicted $\heartsuit$ attack	
		+	-
$\heartsuit$ attack	+	TP	FN
	-	FP	TN

true / false $\begin{matrix} \nearrow + \\ \searrow - \end{matrix}$

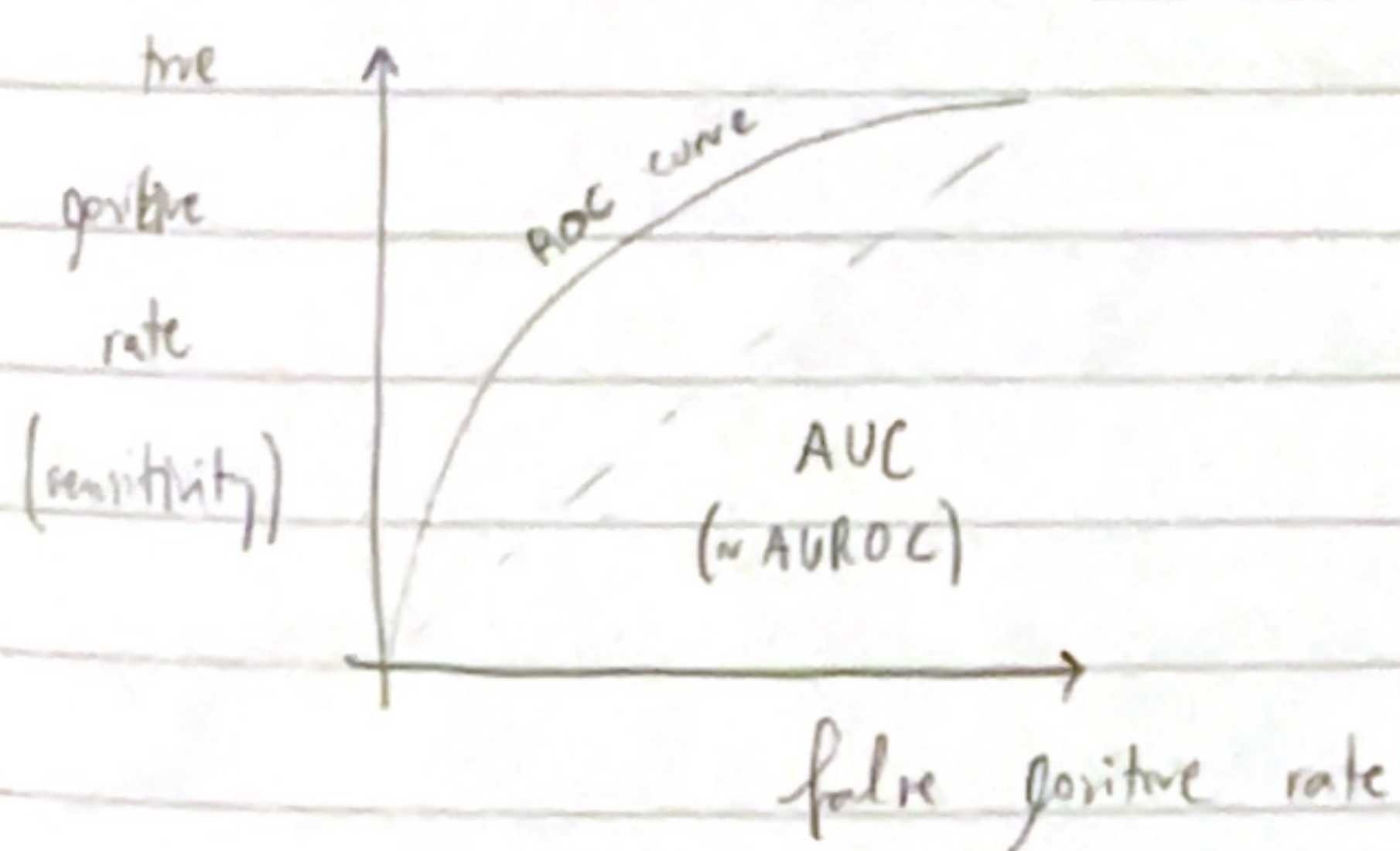
→ Accuracy = $\frac{\text{all correct}}{\text{total data}}$ (0-1)

→ Sensitivity = $\frac{+ \text{ predictions}}{\text{out of all } +}$
or recall

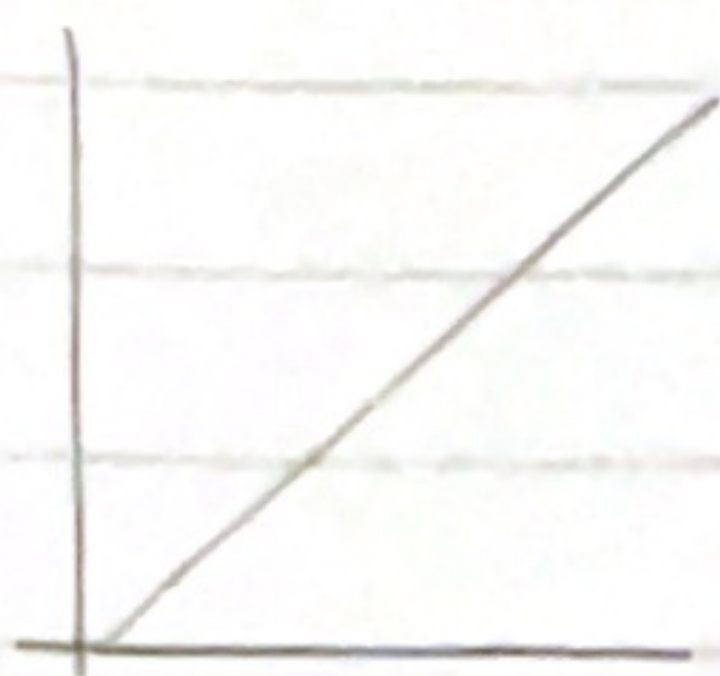
→ Specificity = $\frac{- \text{ predictions}}{\text{out of all } -}$

→ Precision (positive predictive value) → % of correct predicting + } influence by prevalence
→ NPP → % " " " - }

Receiver operator and Precision Recall Curves as evaluation metrics



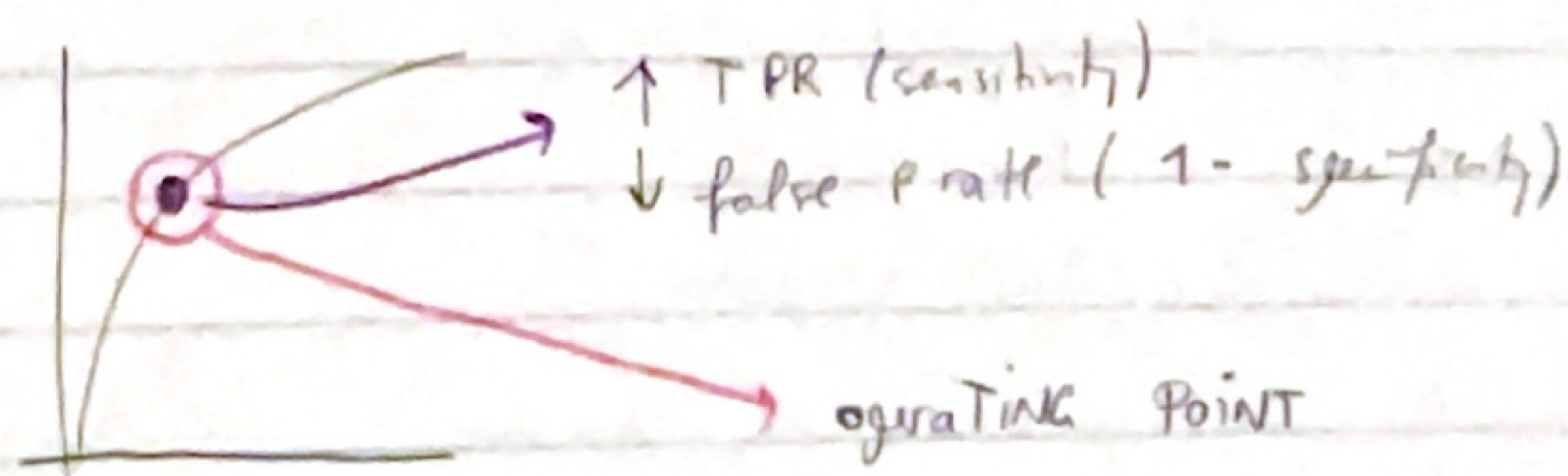
random classifier



(1 - specificity)
perfect classifier



"Good classifier"



Choosing operating point

→ If I prefer ↑ sensitivity (eg. screening) → ↑ False Positive Rate

→ If I - ↑ specificity →

By choosing misclassification → Choose operating point → minimizes $\sum \text{false } \oplus + \text{false } \ominus$

By optimizing utility → assign a $\neq$ utility value for each type of outcome and optimize total utility

subjective

→ Shape of the curve can be ↑ useful than the area

→ Understanding if the true positive labels are very $\neq$ than n of true negative labels

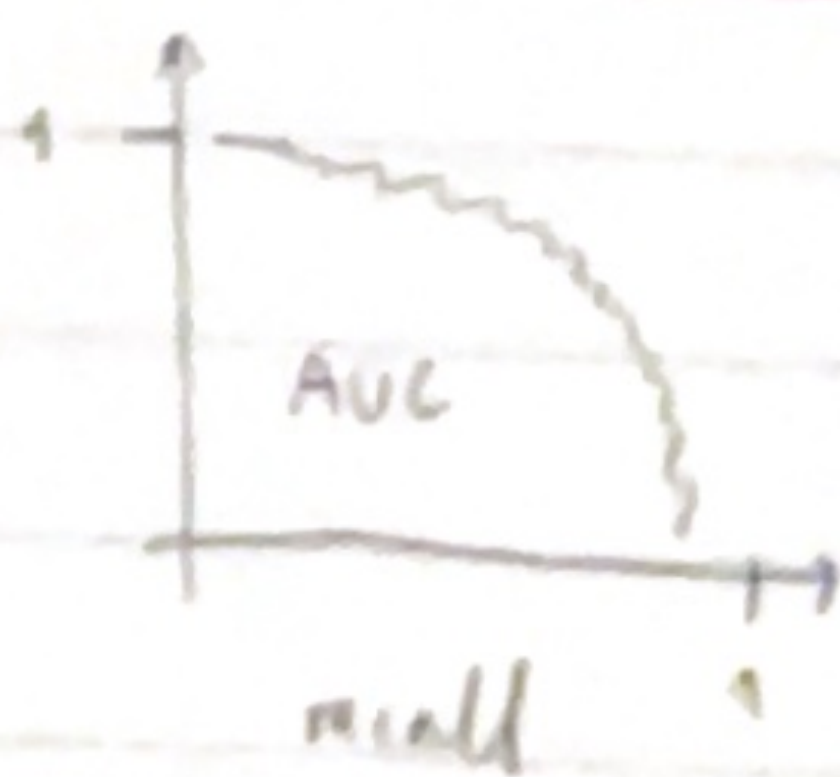
IMBALANCED DATASETS

Alternative to precision

ROC

Precision-recall

curves



don't need/use

big $\neq$

true negative results